

Experiment No-01: Introduction to Structure in C++

Objectives

- Introduce with structure in C++.
- Learn how to use pointers and structure together.
- Learn how to use the structure with functions.

Example 1: Write a C++ to define a structure.

```
#include<iostream>
#include <string>
using namespace std;

// create struct with person1 variable
struct Person {
    string name;
    int citNo;
    float salary;
} person1;

int main() {

    // assign value to name of person1
    getline(cin, person1.name);

    strcpy(person1.name, "Ronaldo");

    // assign values to other person1 variables
    person1.citNo = 1985;
    person1.salary = 2500;

    // print struct variables
    cout<<"Name: "<< person1.name<<endl;
    cout<<"Citizenship No.: "<< person1.citNo<<endl;
    cout<<"Salary: "<< person1.salary;

    return 0;
}
```

Example 2: Write a C++ program to access structure members using pointers.

```
#include<iostream>
```

```
using namespace std;

struct person
{
    int age;
    float weight;
};

int main()
{
    struct person *personPtr, person1;
    personPtr = &person1;

    cout<<"Enter age: ";
    cin>>personPtr->age;

    cout<<"Enter weight: ";
    cin>>personPtr->weight;

    cout<<"Displaying:\n";
    cout<<"Age: "<< (*personPtr).age<<endl;
    cout<<"weight: "<< personPtr->weight<<endl;

    return 0;
}
```

Example 3: Write a C++ program to pass structs to a function.

```
#include<iostream>
#include <string>
using namespace std;

struct student {
    string name;
    int age;
};

// function prototype
void display(struct student s);

int main() {
    struct student s1;

    cout<<"Enter name: "<<endl;
```

```
// read string input from the user
getline(cin,s1.name);

cout<<"Enter age: "<<endl;
cin>>s1.age;

display(s1); // passing struct as an argument

return 0;
}

void display(struct student s) {
    cout<<"Displaying information"<<endl;
    cout<<"Name: "<< s.name;
    cout<<"\nAge: "<< s.age;
}
```

*** For better understanding please feel free to search on internet because it is the best source of learning. ***

Practice Exercise

1. Write a C++ program to store and print the roll no., name, age, and marks of a student using structures.
2. Write a C++ program to store roll no. (starting from 1), name and age of 5 students and then print the details of the student with roll no. 2.
3. Enter the marks of 5 students in Chemistry, Mathematics, and Physics (each out of 100) using a structure named Marks having elements roll no., name, chem_marks, maths_marks, and phy_marks and then display the percentage of each student.
4. Write a C++ program to add two distances in inch-feet using structure.
5. Write a C++ program to subtract two complex numbers.

Experiment No-02: Stack, Queue, and Vector in C++.

Objectives

- Introduce with stack, queue, and vector container of STL in C++.

Example 1: Vector in C++.

```
/**
vector: Member Functions

1) push_back(element) -----> push_back() is used for inserting an
   element at the end of the vector

2) pop_back() -----> pop_back() is used to remove the last
   element from the vector. It reduces the size of the vector by one.

8) clear() ----- > This method clears the whole vector,
   removes all the elements from the vector but do not delete the
   vector.

9) size() -----> returns the size of the vector

**/

#include<bits/stdc++.h>
using namespace std;

int main()
{
    vector<int>vec1; //int type vector declaration
    vector<string>vec2; // string type vector declaration

    // Push_back operatin on vec1

    for(int i=0;i<5;i++){
        vec1.push_back(i);
    }
    vec1.push_back(100);
    vec1.push_back(10);
    vec1.push_back(23);
    vec1.push_back(9);

    // Print the elements of the vector
    for(int i=0;i<vec1.size();i++){
        cout<<vec1[i]<<"\t";
    }
}
```

Example 2: Stack in C++.

```
#include<bits/stdc++.h>
using namespace std;

// Stack Container in C++

int main() {

stack<int>mystack; // variable declaration

mystack.push(42); // push operation
mystack.push(11);
mystack.push(5);
mystack.push(71);
mystack.push(43);

while(!mystack.empty()){
    cout<<mystack.top()<<" ";
    mystack.pop(); // pop operation
}

}
```

Example 3: Queue in C++.

```
#include<bits/stdc++.h>
using namespace std;

// Queue Container in C++

int main ()
{
    queue <int> q; // creates an empty queue of integer q

    q.push(2); // pushes 2 in the queue , now front = back = 2
    q.push(3); // pushes 3 in the queue , now front = 2 , and back = 3
    q.push(8);
    q.push(45);
    q.push(60);
    q.push(80);

    while(!q.empty()){

        cout<<q.front()<<" ";

        q.pop();
    }

}
```

Practice Exercise

1. Take 5 integer values into a stack. Find the summation of all the stack elements.
2. Take 6 integer values (0 to 5) into a stack and then find the factorial of each stack element. Store the outputs in another stack. Print the output in the following way:
Factorial : 0 = 1
Factorial : 1 = 1
Factorial : 2 = 4

—

Experiment No-03: Sorting Algorithms in C++.

Objectives

- Learn different sorting algorithms and their complexities.
- Implement the insertion sort and selection sort algorithm in C++.

Algorithm: Insertion Sort.

```
InsertionSort (arr[], N)
{
  for i ∈ (1, N) do
    key ← arr[i]
    hole ← i
    while j > 0 & arr[hole − 1] > key do
      arr[hole] ← arr[hole − 1]
      hole ← hole − 1
    end while
    arr[hole] ← key
  end for
}
```

Example 1: A C++ program to implement the Insertion Sort algorithm.

```
#include<iostream>
using namespace std;

int main()
{
  int arr[10] = {5, 1, 4, 2, 8}, i, j, k, value, hole;
  for (i=1; i<5; i++){
    value = arr[i];
    hole = i;
    while(hole>0 && arr[hole-1]>value){
      arr[hole] = arr[hole-1];
      hole = hole-1;
    }
    arr[hole] = value;
  }
  for(k=0; k<5; k++){
    cout<<arr[k]<<" ";
  }
  cout<<endl;
  for(k=0; k<5; k++){
    cout<<arr[k]<<" ";
  }
  return 0;
}
```

Algorithm: Selection Sort.

```
SelectionSort ( $arr[]$ ,  $N$ )
{
  for  $i \in (0, N)$  do
     $min \leftarrow i$ 
    for  $j \in (i + 1, N)$  do
      if  $arr[j] < arr[min]$  then
         $min \leftarrow j$ 
      end if
    end for
    swap( $arr[j]$ ,  $arr[min]$ )
  end for
}
```

Example 2: A C++ program to implement the Selection Sort algorithm.

```
#include<iostream>
using namespace std;

int main()
{
  int arr[10] = {77, 33, 44, 11, 88, 22, 66, 55}, n=8, i, j, k;

  for (i=0; i<n; i++){
    int min_index = i;

    for (j=i+1; j<n; j++){
      if (arr[j]<arr[min_index])
      {
        min_index = j;
      }
    }
    cout<<"Pass: "<<i+1 <<" " <<" --> " << min_index+1 <<endl;
    swap(arr[i], arr[min_index]); // swap value
  }
  for (k=0; k<n; k++){
    cout<< arr[k]<<" ";
  }
  cout<<endl;

  return 0;
}
```

Practice Exercise

1. Implement the Insertion sort algorithm using a function in C++ (Argument passed and no Return value).
2. Implement the Selection Sort algorithm using a function in C++ (Argument passed and no Return value).

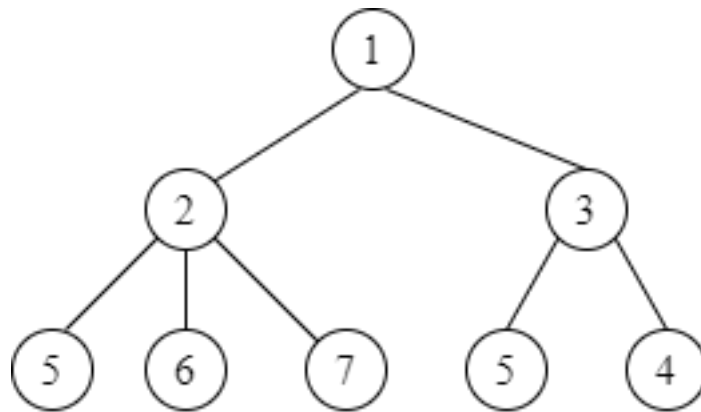
[Insertion Sort Video](#)

Experiment No-04: Tree Representation in C++.

Objectives

- Represent a tree using adjacency list.
- Find depth, height and parent nodes of a tree.

Example 1: Tree representation using adjacency list in C++.



```
#include<bits/stdc++.h>
using namespace std;

vector<int> adj[100];
vector<int> lst; // to store the unique tree nodes
int vis[100000]; // initialize all the elements to zero

int check(int v)
{
    if(vis[v]==0)
    {
        lst.push_back(v);
        vis[v]=1;
    }
}

int main() {
    int i, j, k;
    int n;
    cout<<"Number of Nodes: "<<endl;
    cin >> n; // number of nodes
    cout<<"Enter the edge connections: "<<endl;
    for (i = 0; i < n-1; ++i) {
        int u, v;
        cin >> u >> v; // edge connections
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
}
```

```

        // check the node is visited or not
        check(u);
        check(v);
    }

    // print the tree connection

    cout <<"Tree Connections: " <<endl;
    for(i=0;i<lst.size();i++)
    {
        k=lst[i];
        cout<<"Node : "<<k<<"--> ";
        for(j=0;j<adj[k].size();j++)
            cout<<adj[k][j]<<' ';
        cout<<endl;
    }
}

```

Example 2: Find depth, height and parent nodes of a tree in C++.

```

#include<bits/stdc++.h>
using namespace std;

vector<int> adj[100];
vector<int> lst;
int par[100], depth[100], vis[100000];

void traverse(int node, int p) {

    par[node] = p; // assign parent
    if (p == -1) {
        depth[node] = 0;
    } else {
        depth[node] = depth[p]+1;
    }
    for (int i = 0; i < adj[node].size(); ++i) {
        int nxt = adj[node][i];
        if (nxt == p) {
            continue; // if already traverse then skip
        }
        traverse(nxt, node); //recursively called
    }
}

int check(int v)
{
    if(vis[v]==0)
    {
        lst.push_back(v);
        vis[v]=1;
    }
}

```

```
int main() {
    int i, j, k;
    int n;
    cout<<"Number of Nodes: "<<endl;
    cin >> n; // number of nodes
    cout<<"Enter the edge connections: "<<endl;
    for (i = 0; i < n-1; ++i) {
        int u, v;
        cin >> u >> v; // edge connections
        adj[u].push_back(v);
        adj[v].push_back(u);
        check(u);
        check(v);
    }
    int root = lst[0];
    traverse(root, -1);

    int mx = depth[0];
    for (int u = 0; u < lst.size(); ++u) {
        i = lst[u];
        cout <<"Node: "<< i << " "<<"Parent: " << par[i] << " " <<
            "Depth: "<< depth[i] << endl;
        mx = max(mx, depth[i]);
    }
    cout<<"Height of the Tree: "<<mx<<endl;
}
```

Practice Exercise

1. Represent the Trees shown in Figure 1 using adjacency list in C++.
2. Find the depth (each node), height and parent nodes of the Trees.

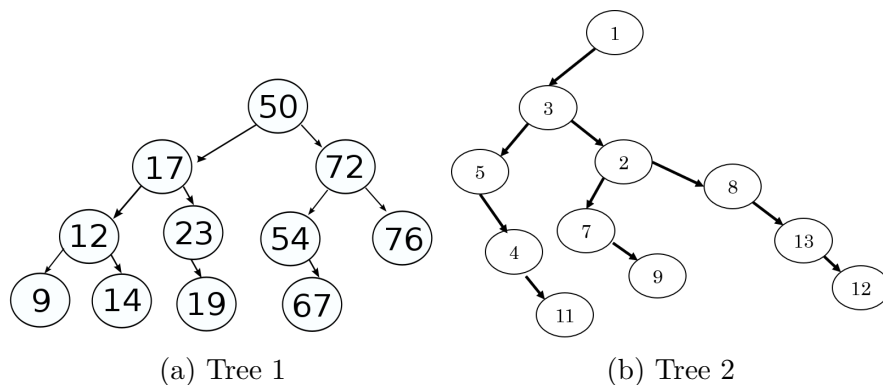


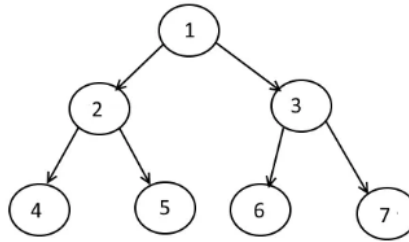
Figure 1

Experiment No-05: Binary Tree Traversal using Recursion in C++.

Objectives

- Introduce Tree traversal techniques.
- Find Inorder, Preorder, and Postorder traversal.

Example 1: Inorder traversal using recursion.



```
#include<bits/stdc++.h>
using namespace std;

int lft[100], rht[100];
// Traversal function
void Inorder(int node) {
    if (node == -1) {
        return;
    }
    Inorder(lft[node]);
    cout << node << " ";
    Inorder(rht[node]);
}

int main() {
    int i, j, k, r;
    int n;
    cout<<"Number of nodes:"<<endl;
    cin >> n;
    cout<<"Enter nodes: "<<endl;
    for (i = 1; i <=n; ++i) {
        cout<<"Root: ";
        cin>>r;
        cout<<"L "<<" R " <<endl;
        cin >> lft[r] >> rht[r];
    }
    int root = 1;
    cout<<"\nInorder Traversal: "<<endl;
    Inorder(root); //recursively called
    cout << endl;
}
```

Example 2: Preorder traversal using recursion..

```
#include<bits/stdc++.h>
using namespace std;

int lft[100], rht[100];
// Traversal function
void Preorder(int node) {
    if (node == -1) {
        return;
    }
    cout << node << " ";
    Preorder(lft[node]);
    Preorder(rht[node]);
}

int main() {
    int i, j, k, r;
    int n;
    cout<<"Number of nodes:"<<endl;
    cin >> n;
    cout<<"Enter nodes: "<<endl;
    for (i = 1; i <=n; ++i) {
        cout<<"Root: ";
        cin>>r;
        cout<<"L "<<" R " <<endl;
        cin >> lft[r] >> rht[r];
    }
    int root = 1;
    cout<<"\nPreorder Traversal: "<<endl;
    Preorder(root);
    cout << endl;
}
```

Practice Exercise

1. Write a C++ program to find the Inorder, Preorder, and Postorder traversals of the following trees.

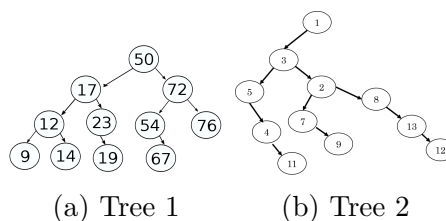


Figure 1

Experiment No-06: Binary Search Tree in C++.

Objectives

- Implement the BST in C++.
- Find Inorder, Preorder, and Postorder traversal.

Example: Implementation of BST.

```
#include<bits/stdc++.h>
using namespace std;

int lft[100], rht[100], val[100], tot_node = 0;

// Tree construction
void Insert(int root, int v) {
    if (val[root] < v) {
        if (rht[root] != -1) {
            Insert(rht[root], v);
        } else {
            tot_node++;
            rht[root] = tot_node;
            val[tot_node] = v;
        }
    } else {
        if (lft[root] != -1) {
            Insert(lft[root], v);
        } else {
            tot_node++;
            lft[root] = tot_node;
            val[tot_node] = v;
        }
    }
}

// Traverse method
void Inorder(int node) {
    if (node == -1) {
        return;
    }
    Inorder(lft[node]);
    cout << val[node] << " ";
    Inorder(rht[node]);
}

int main() {
    int i, j, k;
    int n;
    cin >> n;
    int root = -1;
```

```
// initialize all the values -1
memset(lft, -1, sizeof lft);
memset(rht, -1, sizeof rht);
// take the input values
for (i = 0; i < n; ++i) {
    cin >> k;
    if (root == -1) {
        root = tot_node;
        val[root] = k;
    } else {
        Insert(root, k);
    }
}
Inorder(root);
}
```

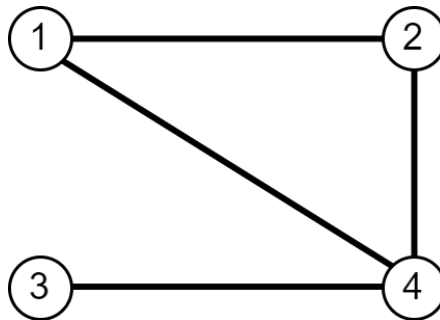
Practice Exercise

1. Write a C++ program to find the Inorder, Preorder, and Postorder traversals of the BST having values = {20, 10, 3, 2, 7, 9, 80, 100, 65, 43, 21, 47, 32, 21}.
2. Write a C++ program to check a Tree is BST or not.

Experiment No-07: Graph Traversal Using Breadth-First Search Technique.

Objectives

- Introduce the Breadth First Search (BFS) technique.
- Find the Shortest Distances Using BFS.
- Find the Shortest Path from Source to Destination .



Example 1: BFS traversal for finding the shortest distance from the source to all the nodes.

```
#include<bits/stdc++.h>
using namespace std;

vector<int> adj[100];
int dis[100], visited[100];

// BFS function
void Bfs(int source) {
    queue<int> q; // declare a empty queue
    dis[source] = 0;
    visited[source] = 1;
    q.push(source); // push source node into queue
    while (!q.empty()) {
        int node = q.front(); // front element of the queue
        for (int i = 0; i < adj[node].size(); ++i) {
            int nxt_node = adj[node][i];
            // already visited then skip
            if (visited[nxt_node]) continue;
            dis[nxt_node] = 1 + dis[node];
            visited[nxt_node] = 1;
            q.push(nxt_node); // push into the queue
        }
        // pop the node
        q.pop();
    }
}
```

```
int main() {
    int i, j, k;
    int n, m;
    cout<< "No.of Nodes: "<<endl;
    cin >> n;
    cout<< "No.of Edges: "<<endl;
    cin >> m;
    cout<<"Enter the edge connections: "<<endl;
    for (i = 0; i < m; ++i) {
        int u, v; // edge inputs
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    int source;
    cout<<"Enter the Source Node: "<<endl;
    cin >> source;
    // call the BFS method
    Bfs(source);
    for (i = 1; i <= n; ++i) {
        cout << "Distance " << source << " to " << i << " = " << dis[i]
            << endl;
    }
}
```

Example 2: Finding the shortest path from a source to destination node.

```
#include<bits/stdc++.h>
using namespace std;

vector<int> adj[100];
vector<int> path;
int parent[100], dis[100], visited[100];

// Function for finding the shortest path
void shortest_path(int d){
    if (d!=-1){
        int p = parent[d];
        path.push_back(d); // push the paths into a vector
        shortest_path(p); // recursively called
    }
}

// BFS function for finding the shortest distance
void Bfs(int source) {
    queue<int> q; // declare a empty queue
    dis[source] = 0;
    visited[source] = 1;
    parent[source] = -1;
    q.push(source); // push source node into queue
    while (!q.empty()) {
```

```

    int node = q.front(); // front element of the queue
    for (int i = 0; i < adj[node].size(); ++i) {
        int nxt_node = adj[node][i];
        // already visited then skip
        if (visited[nxt_node]) continue;
        dis[nxt_node] = 1 + dis[node];
        visited[nxt_node] = 1;
        parent[nxt_node] = node;
        q.push(nxt_node); // push into the queue
    }
    // pop the node
    q.pop();
}

}

int main() {
    int i, j, k;
    int n, m;
    cout<< "No.of Nodes: "<<endl;
    cin >> n;
    cout<< "No.of Edges: "<<endl;
    cin >> m;
    cout<<"Enter the edge connections: "<<endl;

    for (i = 0; i < m; ++i) {
        int u, v; // edge inputs
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int source,dest;
    cout<<"Enter the Source Node: "<<endl;
    cin >> source;
    cout<<"Enter the Destination Node:"<<endl;
    cin>> dest;
    // call the BFS method
    Bfs(source);
    cout<<"Shortest Distance from "<<source<<" to "<<dest<<" =
        "<<dis[dest]<<endl;
    cout<<"Shortest Path is: ";
    shortest_path(dest); // call the shortest path function
    // Reverse the path vector
    reverse(path.begin(), path.end());

    for (k =0;k<path.size(); k++){
        cout<<path[k]<<" ";
    }

}

```

Practice Exercise

Write a C++ program for the following graphs -

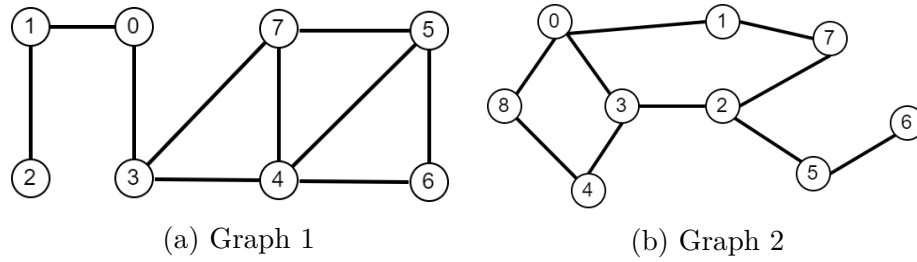


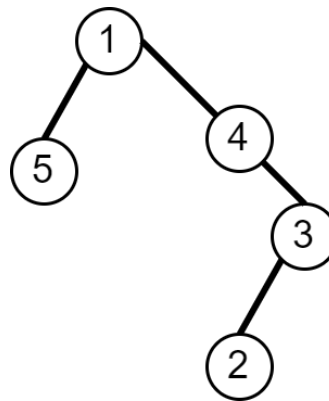
Figure 1

1. To find the shortest distance from an arbitrary source to all the nodes .
2. To find the shortest distance and path from an arbitrary source to an arbitrary destination node.

Experiment No-8: Depth First Search (DFS) Algorithm in C++.

Objectives

- Introduce the Depth First Search (DFS) technique.
- Find the Shortest Distances between Tree nodes Using DFS.
- Find the Shortest Path from Source to Destination.



Example 1: DFS traversal for finding the shortest distance from the source to all the nodes.

```
#include<bits/stdc++.h>
using namespace std;

vector<int> adj[100];
int dis[100], visited[100];

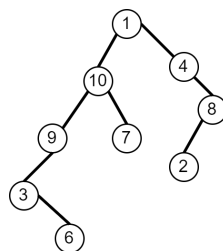
void Dfs(int source) {
    stack <int> st; // declare an empty stack
    dis[source] = 0;
    visited[source] = 1;
    st.push(source); // push source node into stack
    while (!st.empty()) {
        int node = st.top(); // top element of the stack
        st.pop(); // pop the element
        for (int i = 0; i < adj[node].size(); ++i) {
            int nxt_node = adj[node][i];
            if (visited[nxt_node]) continue;
            dis[nxt_node] = 1 + dis[node];
            visited[nxt_node] = 1;
            st.push(nxt_node); // push into stack
        }
    }
}
```

```
int main() {
    int i, j, k;
    int n, m;
    cout<< "No.of Nodes: "<<endl;
    cin >> n;
    cout<< "No.of Edges: "<<endl;
    cin >> m;
    cout<<"Enter the edge connections: "<<endl;

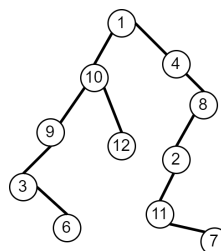
    for (i = 0; i < m; ++i) {
        int u, v; // edge inputs
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    int source;
    cout<<"Enter the Source Node: "<<endl;
    cin >> source;
    // call the DFS method
    Dfs(source);
    for (i = 1; i <= n; ++i) {
        cout << "Distance " << source << " to " << i << " = " << dis[i]
            << endl;
    }
}
```

Practice Exercise

Write a C++ program for the following Graphs -



(a) Graph 1



(b) Graph 2

Figure 2

1. To find the shortest distance from an arbitrary source to all the nodes .
2. To find the shortest distance and path from an arbitrary source to an arbitrary destination node.